

Agenda

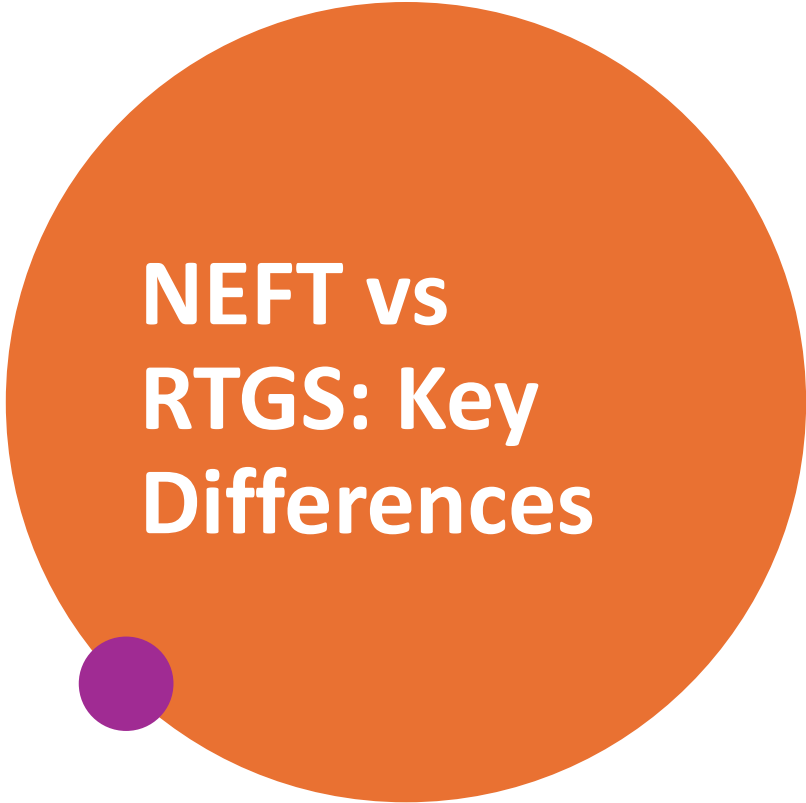
- This Week
 - Class 8 – September 23rd (2:00– 3:00, 3:30 – 4:30)
 - Class 9 – September 24th (10:30 – 12:30)
 - Class 10 – September 25th (1:00 – 3:00)
- Starter Quiz
- Core Banking Platform Engineering : Processing Models
- Thought Paper Presentation - OCEN (Open Credit Enablement Network) – India (Team 2) - 30 mins
- Core Banking Platform Engineering : Processing Models (Cont'd)

Note:

- Mobile Phones – Do not bring to class or Completely Switch off
- If students have any specific topic they want to discuss, share the topic and we can try to schedule
- Please be seated by your teams

Starter Quiz

- What are the 4 Key differences between NEFT and RTGS
 - Speed, Settlement Method, Minimum Transaction Amount, Charges



NEFT vs RTGS: Key Differences

- Speed:**

NEFT takes longer as transactions are processed in batches, typically taking 30 mins to an Hour.

RTGS provides instant transfers, with funds reaching the beneficiary's account almost immediately

- Settlement Method:**

NEFT processes transactions in batches at scheduled intervals. RTGS processes each transaction individually and in real-time, ensuring immediate finality.

- Minimum Transaction Amount:**

RTGS has a minimum transaction amount of ₹2 lakh.

NEFT does not have a minimum limit.

- Charges:**

If it is done online (i.e. self-service channel), both NEFT and RTGS are free. However, if the money is transferred by visiting the bank branch, NEFT is cheaper than RTGS

Starter Quiz

- What does "ACID" mean in the context of "Transaction Principle" and a brief description of each Alphabet in "ACID"

ACID Transaction Principle

Atomicity

Atomicity ensures all database operations in a transaction are completed successfully, or none are applied at all.

Consistency

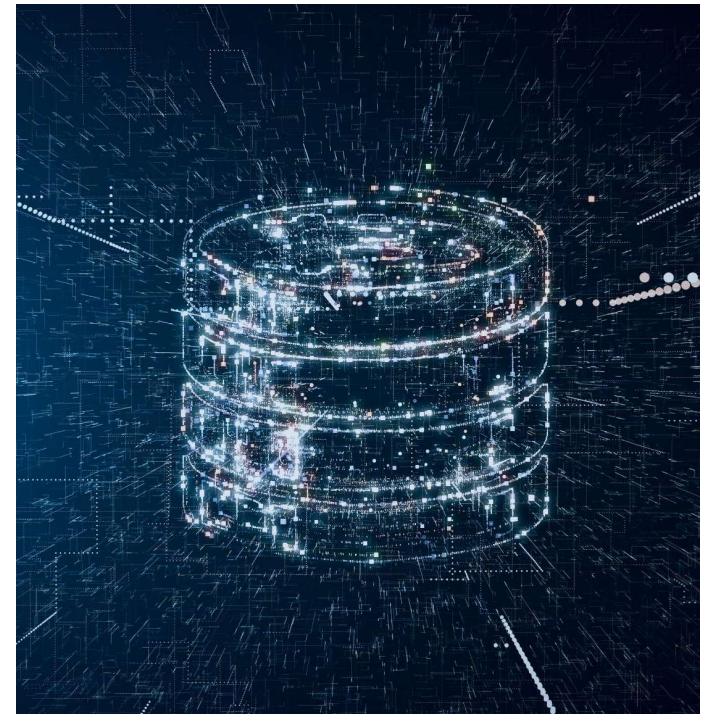
Consistency preserves the integrity of database information before and after transactions, maintaining valid data states.

Transaction Isolation

Isolation prevents concurrent database transactions from affecting each other's results, ensuring accuracy and reliability.

Durability

Durability means that once a transaction is committed, its changes remain safe even after failures.



Core Banking Platform Engineering - Processing Models



Processing Models in Core Banking

- **Real Time Processing:** For scenarios where immediate action and response is needed with low latency.
- **Near Real Time Processing:** For scenarios that can doesn't require the low latency of true real-time but also cannot wait until end of day.
- **Batch Processing:** For Periodic or high-volume tasks where data can be processed together at set times, often leading to simpler and more cost-effective solutions.

Processing Models Comparison

MODEL	SPEED	USE CASE	COST
Batch	Scheduled	Interest, Scheduled EOD & BOD Events, Reports & Extracts ..	Low
Real Time	Instant	Payments, Account Management ..	High
Near Real Time	Reactive	Customer Notifications, DW Ingestion ..	Moderate

Sample Lab Business Processes – Relating to Processing Models

- **Business Process Design:** When Modelling a Business Process one of the actions is to identify what processing models should be used to most Optimally Implement the Business Process keeping in view of the architecture of the components involved in fulfilling the Business Process
- **Term/Fixed Deposit Processes:**
 - **Account Open:**
 - **Real Time:**
 - Simulation
 - Account Open
 - Initial Funding
 - **Near Realtime:**
 - Customer Communications
 - Alert
 - Notice
 - **Batch Processes:**
 - Maturity Process
 - Statements
 - Interest Accruals
 - Reports (Account Open List Report)

Historical evolution of banking transaction processing



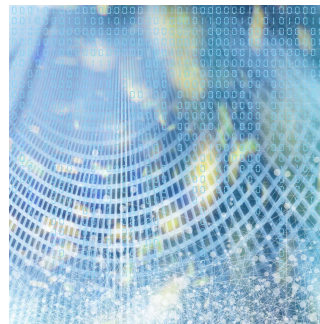
Manual Ledger Entries

Early banking transactions were recorded manually in physical ledgers, requiring extensive human effort and time.



Batch Computer Processing

Introduction of batch processing automated transaction handling, improving speed but not offering instant updates.



Real-time Transaction Systems

Advanced computing enabled real-time processing, allowing instant transaction updates and improved customer service.

Real-Time Processing Pattern Overview

Note:

API Versioning, Backward Compatibility, API Gateway concepts will be covered in Integration Architecture session

API Security aspects will be covered in Security Architecture session

API Performance aspects will be covered in the Performance Architecture Session



Why Real time Processing - Financial Transactions

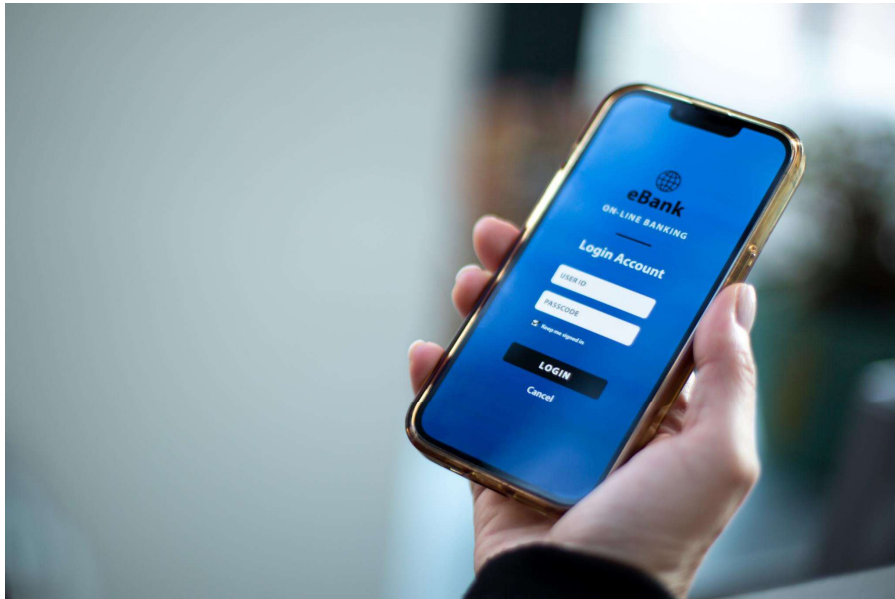
Instant Money Transfers

Customers send money instantly across accounts and borders, significantly reducing delays and enhancing liquidity.

Real-time Payment Convenience

Real-time payments improve convenience and cater to modern commerce needs with efficient transaction processing.

Why Real time Processing - Account Management



- Instant Account Updates
 - Banks offer real-time notifications to help customers monitor account management updates.
- Empowered Financial Decisions
 - Timely account information empowers customers to make smarter financial choices and manage money efficiently.



Definition and key characteristics of real-time processing

Immediate Data Processing

Real-time processing handles data instantly as transactions happen, ensuring up-to-date accuracy.

Continuous Availability

Systems operate continuously to provide uninterrupted data processing and availability.

Minimal Latency

Processing occurs with minimal delay, ensuring responsive and timely data handling.

Instant User Feedback

Users receive immediate responses from the system, enhancing interaction and decision-making.

Infrastructure Complexity

Real-time processing demands more infrastructure and robust observability mechanisms.



Implementing Real time processing - Application Programming Interfaces (APIs)

Seamless Communication

APIs enable smooth data exchange between banking systems and external services in real time.

Instant Payment Processing

APIs facilitate fast and secure payment and account management transactions.

Service Integration

APIs support integration of diverse financial services, enhancing user experience and functionality.

Introduction to API Protocols

REST - REpresentational State Transfer

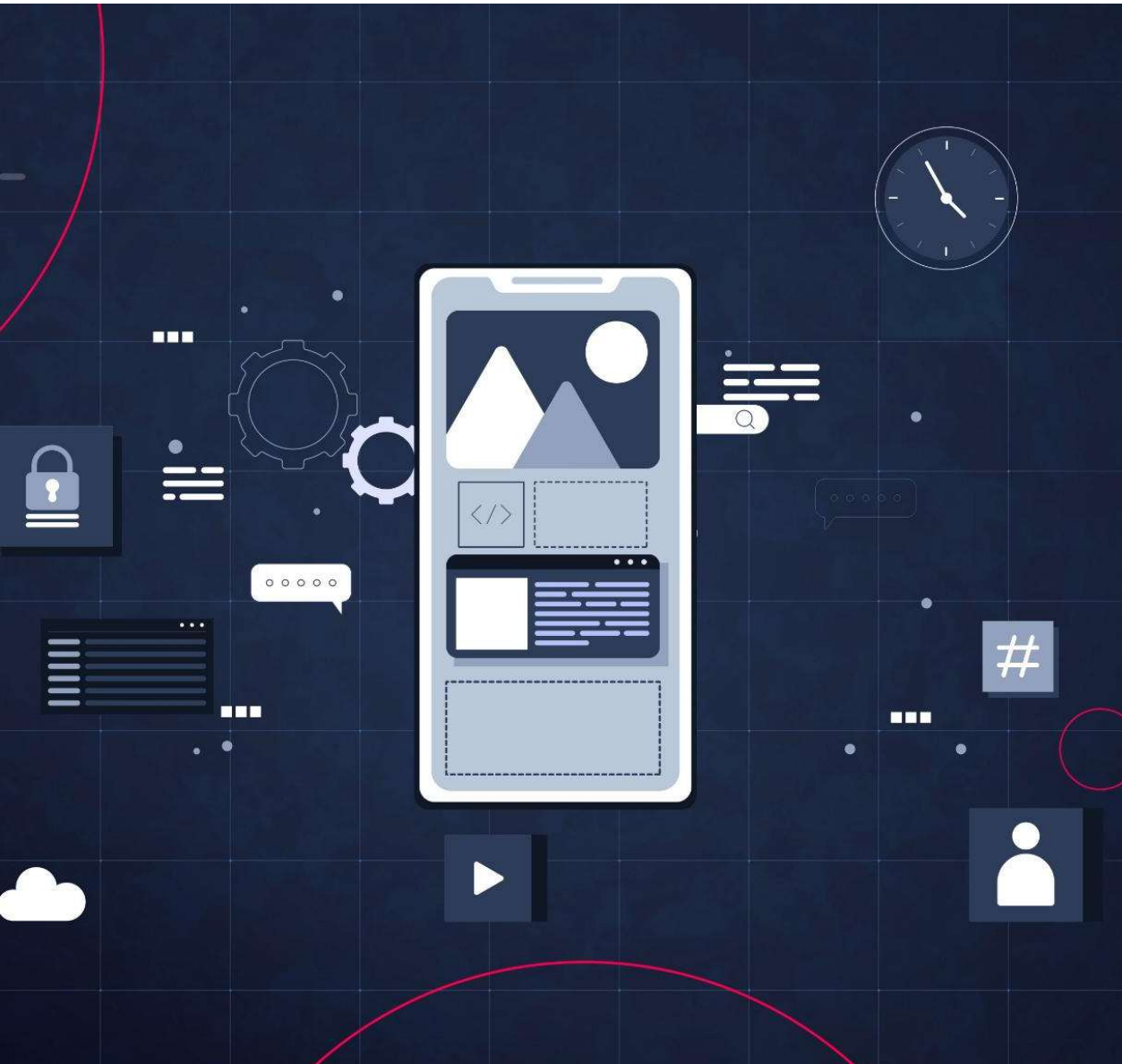
REST uses HTTP/1.1 and JSON for data exchange, offering simplicity and easy debugging for APIs. Widely used for Integration between various Components.

gRPC – Remote Procedural Call initially created by Google

gRPC uses Protocol Buffers over HTTP/2, enabling high performance and used for integration between apps within a Component.

SOAP – Used in Legacy Apps and is not a focus for this class

Utilizes XML to encode messages, ensuring platform independence



WHAT IS REST?

REST Architectural Style

REST defines constraints for web services based on a stateless, client-server model ensuring each request has all necessary information.

HTTP Methods and Resources

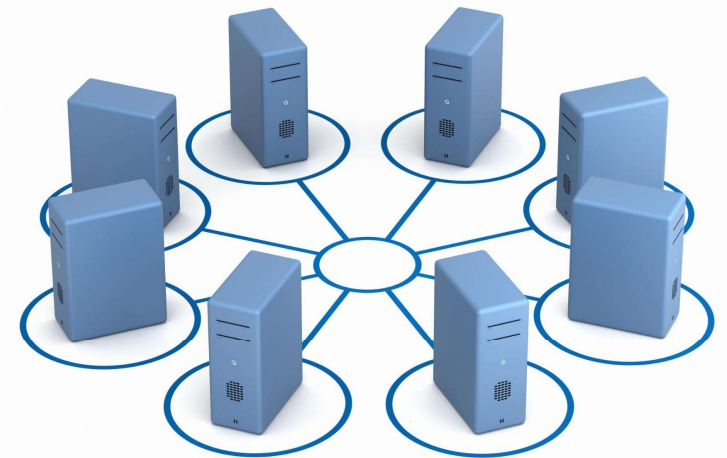
RESTful services use HTTP methods like GET, POST, PUT, DELETE to operate on resources often formatted in JSON.

Advantages of REST

REST is simple, easy to implement, testable, and integrates well with browsers and mobile apps.

Limitations of REST

REST may be slightly inefficient due to HTTP/1.1 and text-based formats.



WHAT IS GRPC?

Efficient Data Serialization

gRPC uses Protocol Buffers for compact data serialization that reduces payload size and boosts performance.

HTTP/2 Features

gRPC operates over HTTP/2 efficient communication.

Performance and Scalability

Offers superior performance and scalability, ideal for microservices and real-time data processing environments.



KEY DIFFERENCES – REST VS GRPC

FEATURE	REST	GRPC
Transport Protocol	HTTP/1.1	HTTP/2
Data Format	JSON	Protocol Buffers
Performance	Moderate	High
Language Support	Broad	Broad (with code generation)
Debugging	Simple	Requires tooling
Use Cases	Web APIs, third-party apps	Microservices, internal APIs



USE CASES AND INDUSTRY ADOPTION

REST Use Cases

REST is widely used for public APIs, mobile apps, and third-party integrations due to simplicity and web compatibility.

gRPC Use Cases

gRPC is favored for internal microservices due to its better performant nature

Industry Adoption

Major companies adopt REST for web services/ public API's and gRPC for internal microservices

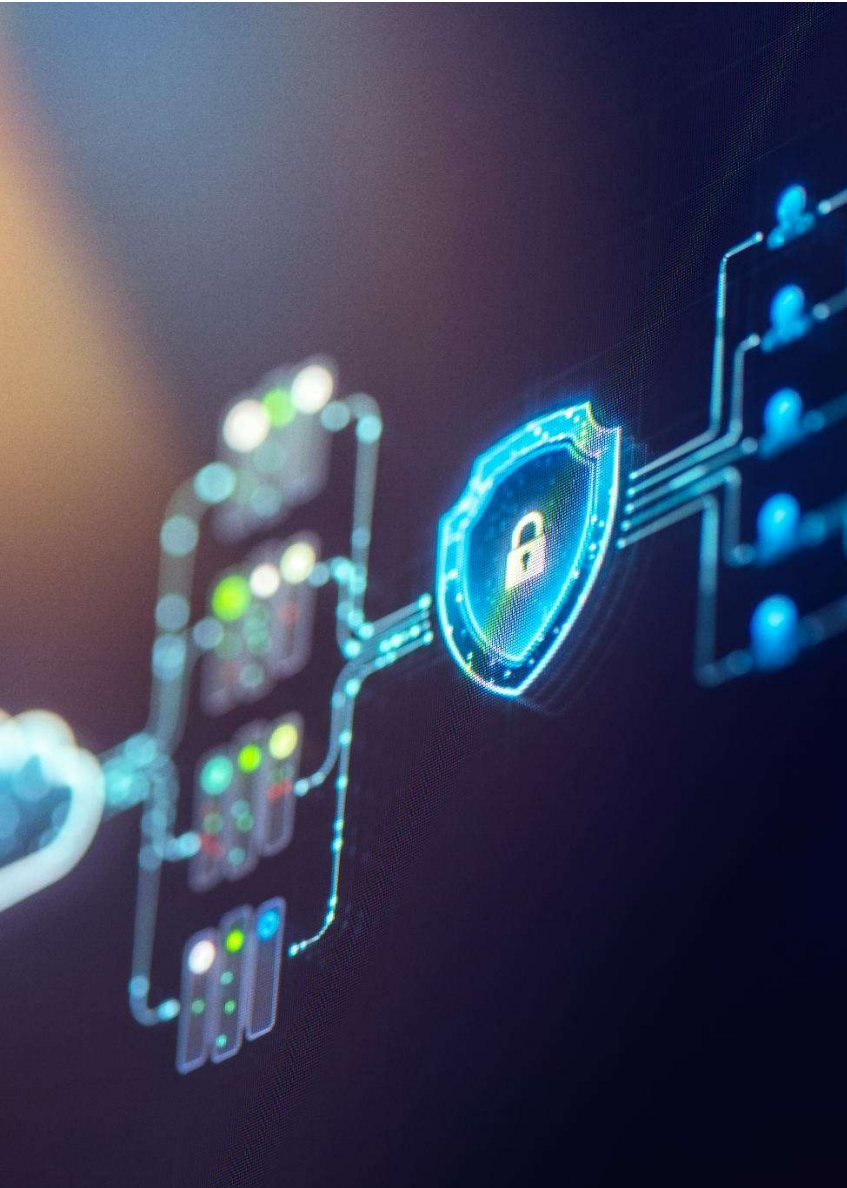
API – Key Real Life Usage Considerations

Note:

Versioning, Backward Compatibility, API Gateway concepts will be covered in Integration Architecture session

Security aspects will be covered in Security Architecture session

Performance aspects will be covered in the Performance Architecture Session



REST API Headers – Basics

Conveying Data Format

Headers specify content type so both client and server understand the data format, ensuring correct processing.

Enhancing Security

Authentication tokens in headers help secure API endpoints and control access to sensitive data.

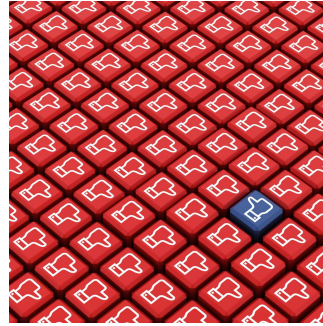
Universally Unique Identifier

UUID refers to the inclusion of a Universally Unique Identifier (UUID) within the HTTP headers of an API request. This UUID serves as a unique identifier for various purposes,

Supporting Custom Requirements

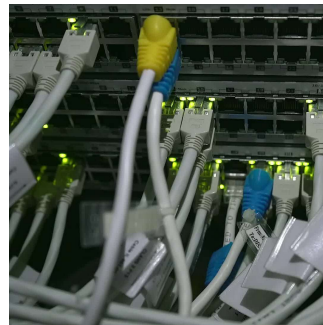
Custom headers enable specialized behavior or features tailored to individual API needs and workflows.

API- Idempotency



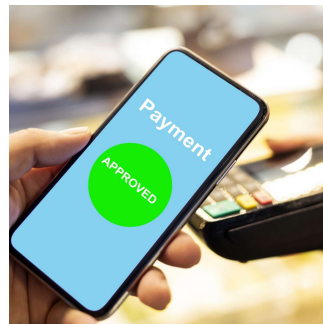
Consistent Results from Requests

Idempotency in APIs means repeating a request always produces the same result, regardless of how many times it is sent.



Handling Network Issues

Idempotency is crucial when network glitches cause duplicate requests, ensuring the system responds safely and reliably.



Preventing Unintended Side Effects

With idempotency, APIs avoid problems like double charges or duplicate data, improving trust and reliability for users.

Idempotency Keys are included in API Header and Validation done using this to prevent Duplicate Processing

API Documentation - OpenAPI Specification (Swagger)

Standardized API Documentation

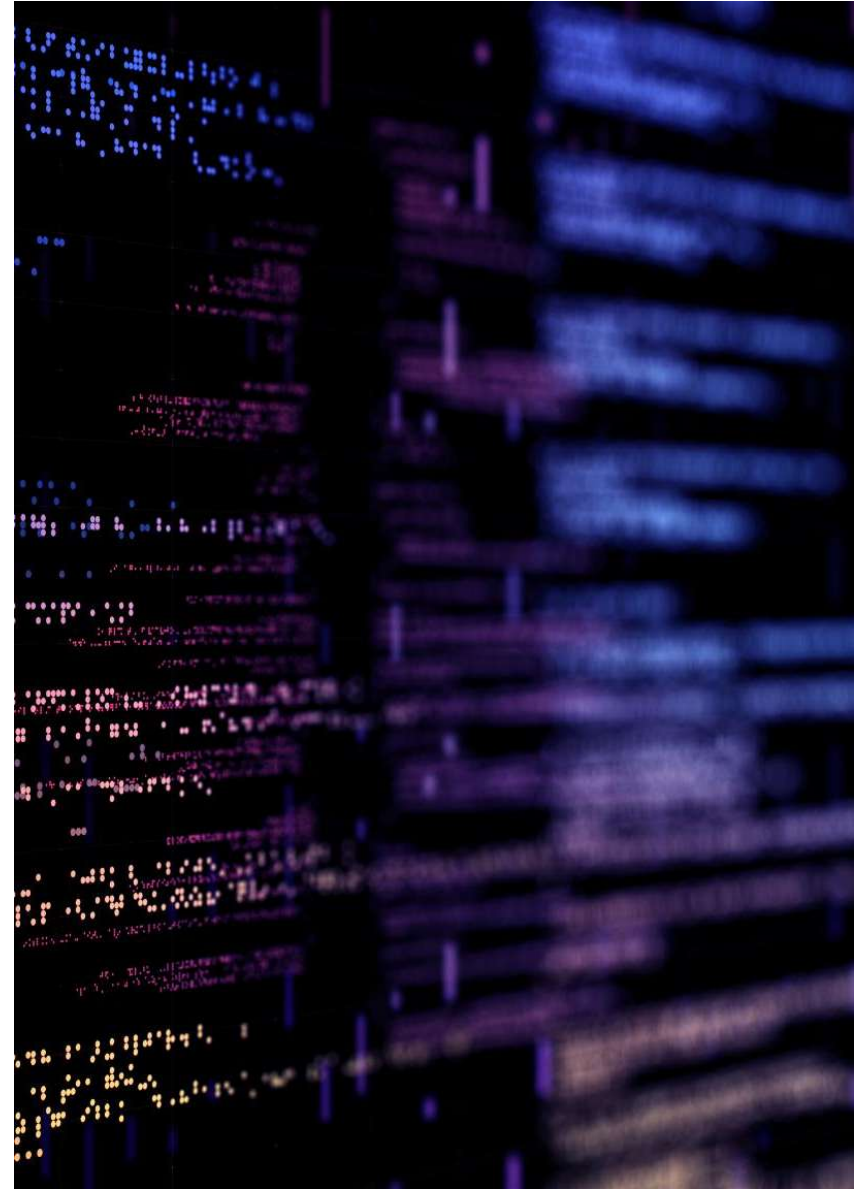
Swagger uses the **OpenAPI Specification** to create machine-readable API definitions, ensuring clarity and consistency.

Interactive API Exploration

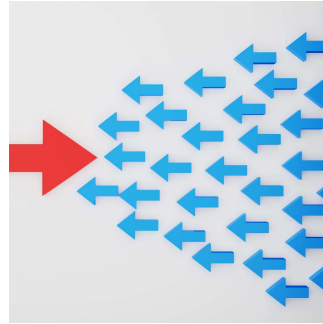
Swagger allows users to interact with APIs directly, making testing and exploration straightforward and efficient.

Automatic Code Generation

Swagger generates client and server code automatically, streamlining the development process for API consumers and providers.



Handling Failures in Micro Services - SAGA Pattern



Decentralized Transaction Management

Saga pattern divides a transaction into smaller, isolated steps managed by different microservices to avoid central coordination.



Compensating Actions for Failures

Each step in a saga has a compensating action, providing a way to undo changes if any failure occurs along the process.



Enhanced Reliability and Consistency

By isolating failures and automating error handling, the saga pattern increases reliability and maintains data consistency across microservices.

Overview of SAGA Pattern Challenges

Complexity of Implementation

Saga pattern introduces complexity due to coordinating multiple local transactions across distributed services.

State Management Challenges

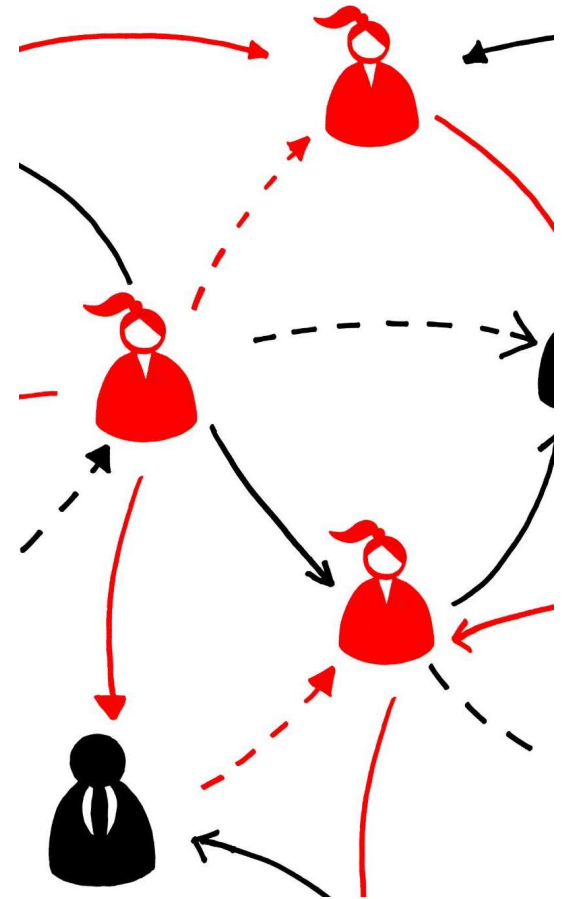
Managing state across services is difficult, requiring careful tracking for consistency and recovery.

Ensuring Eventual Consistency

Maintaining eventual consistency demands effective event coordination and handling of data synchronization delays.

Compensating Transactions Design

Designing compensating transactions is essential to rollback changes when failures occur in distributed workflows.





Real time processing Trends

Growing Transaction Volumes

Increasing transaction volumes require systems that can scale efficiently to handle demand without performance loss.

E.g. UPI Transaction Volume in India to almost triple from 20 Billion transactions per month in August 2025 to 54 Billion by 2030

High Availability and Elasticity

Rise of Digital Natives and Agentic Commerce would demand Banks need reliable, resilient, elastic infrastructure and Core Platforms to ensure continuous uptime and variable performance load.

Open Banking Ecosystems

Expanding open banking networks enable seamless data sharing, increasing service personalization and speed.

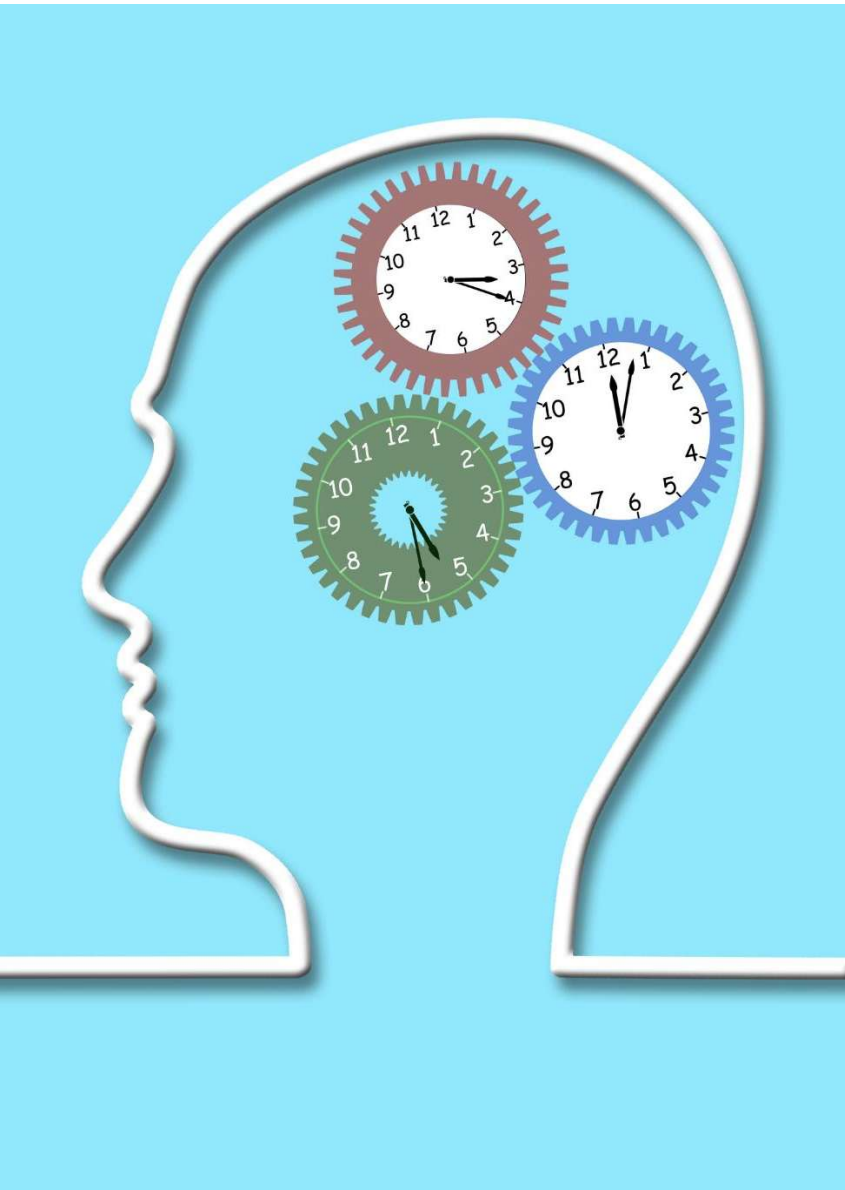
Near Real-Time Processing Pattern Overview

Note:

Event Versioning, Backward Compatibility will be covered in Integration Architecture session

Event Security aspects will be covered in Security Architecture session

Event Performance aspects will be covered in the Performance Architecture Session



Understanding real-time versus near real-time processing

Real-time Processing

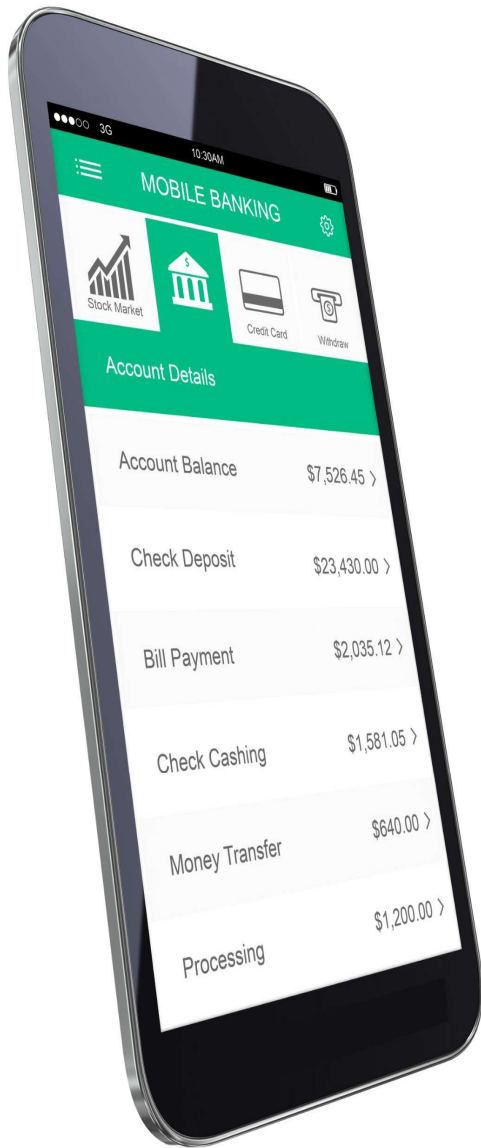
Real-time processing handles transactions immediately without any delay for instant results.

Near Real-time Processing

Near real-time processing involves minimal latency, typically a few seconds to minutes, allowing slight delays.

Balancing Speed and Reliability

Choosing between real-time and near real-time balances processing speed with system reliability and scalability.



Use cases of Near Real time Processing in Core Banking

Customer Communication/Notifications - Real-time Transaction and Account Maintenance Alerts

Notifications about transactions and account maintenance provide customers immediate updates on their financial activities, enhancing security and awareness.

Ingestion into Data Warehouse, Risk Platforms and Enterprise Component's

Data warehouse: Transaction and account maintenance information can be ingested into DW in near real time rather than waiting up to end of day

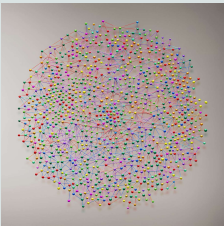
Enterprise Components: Data can be shared to Enterprise Components e.g.. Account Data to Enterprise Customer Component for 360 View

Shifting of certain Processing from Day End to minimize Day End execution time

To Optimize API response time and at the time same time not waiting until Day End to minimize Day End Execution time can be done in Near Real time, this provides a best composition of processing between the 3 models.

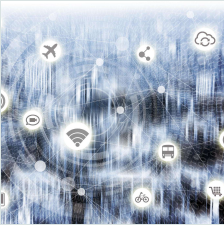
e.g. in TD creation of Next Maturity Event can be done Post New Account Open in near real time processing after the account is opened

NEAR REAL TIME PROCESSING – EVENT DRIVEN ARCHITECTURE OVERVIEW



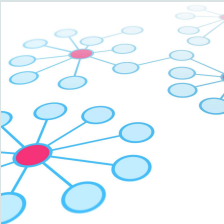
Asynchronous Event Communication

EDA enables system components to produce and consume events asynchronously, improving responsiveness and decoupling.



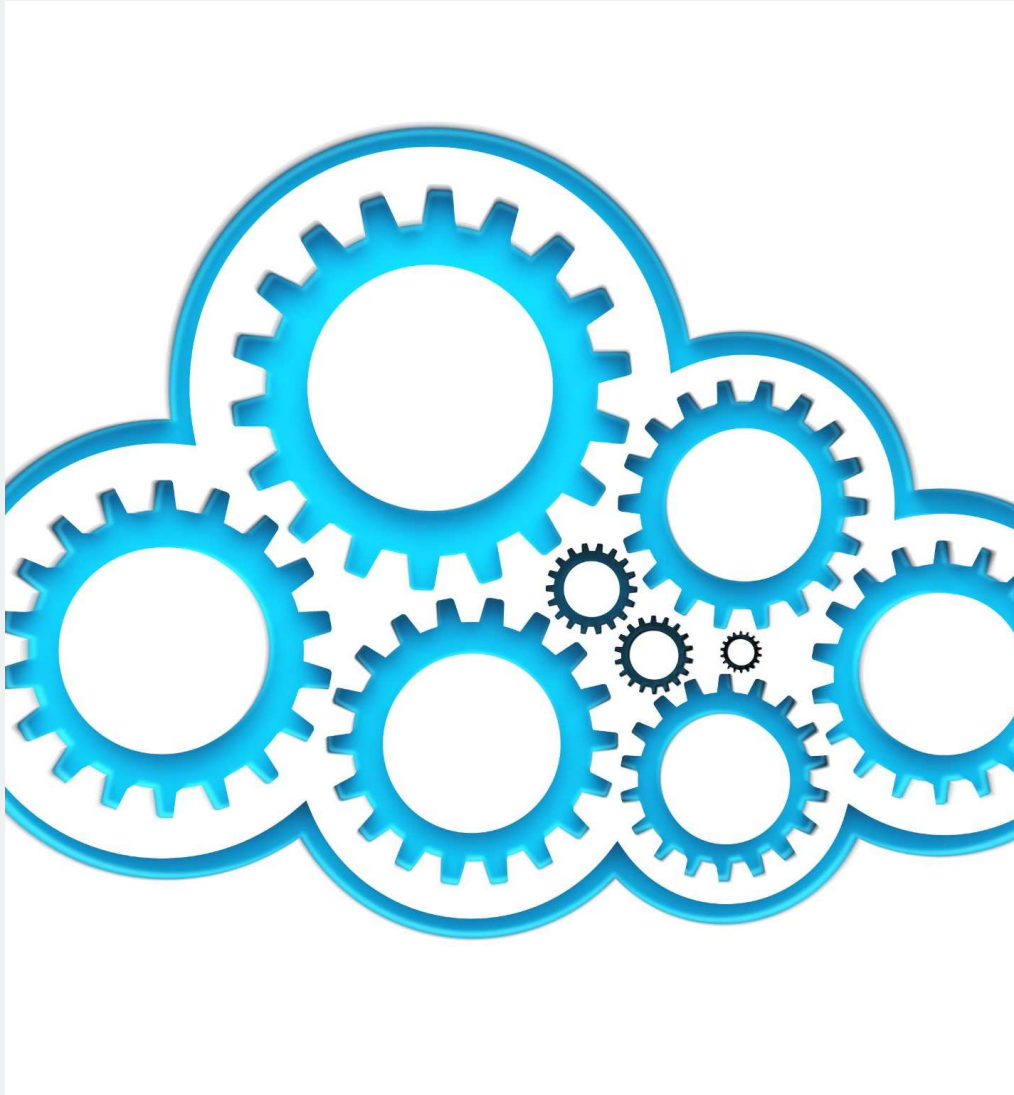
Real-Time Data Processing

Real-time data flow in EDA supports seamless digital banking experiences and rapid integration of new services.



High Availability and Resilience

EDA supports system resilience by allowing components to fail or update independently without system downtime.



EVENT DRIVEN ARCHITECTURE – CORE PRINCIPLES AND PATTERNS

Asynchronous Event Handling

EDA uses asynchronous events to enable flexible and maintainable system interactions without direct calls.

Decoupling of Components

Decoupling allows independent operation of services, reducing dependencies and enhancing scalability.

Design Patterns in EDA

Patterns like Event Partitioning performance and scalability.

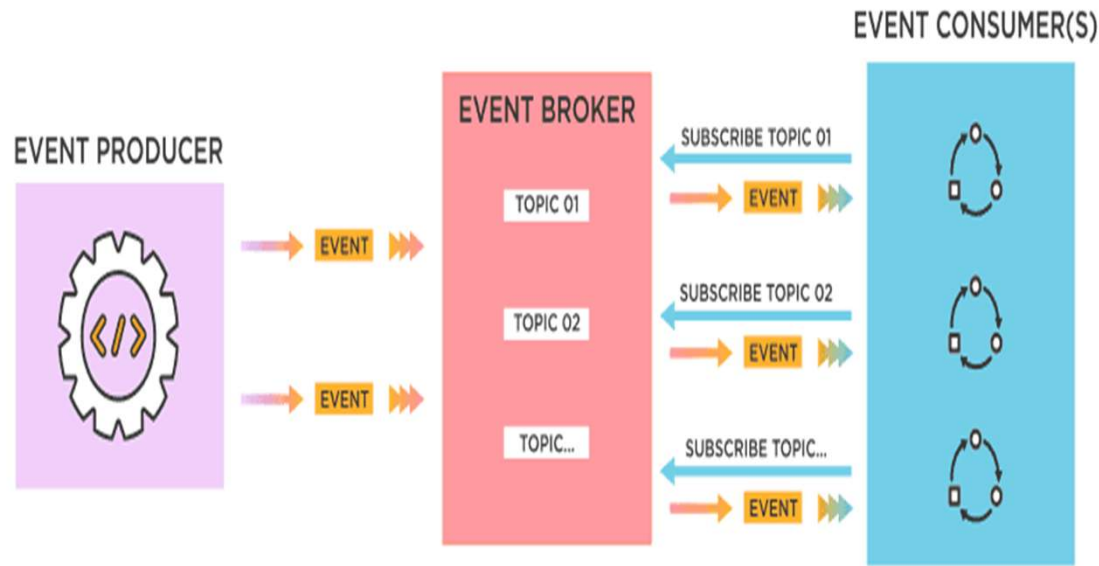
Event Producers & Consumers

Role of Event Producers

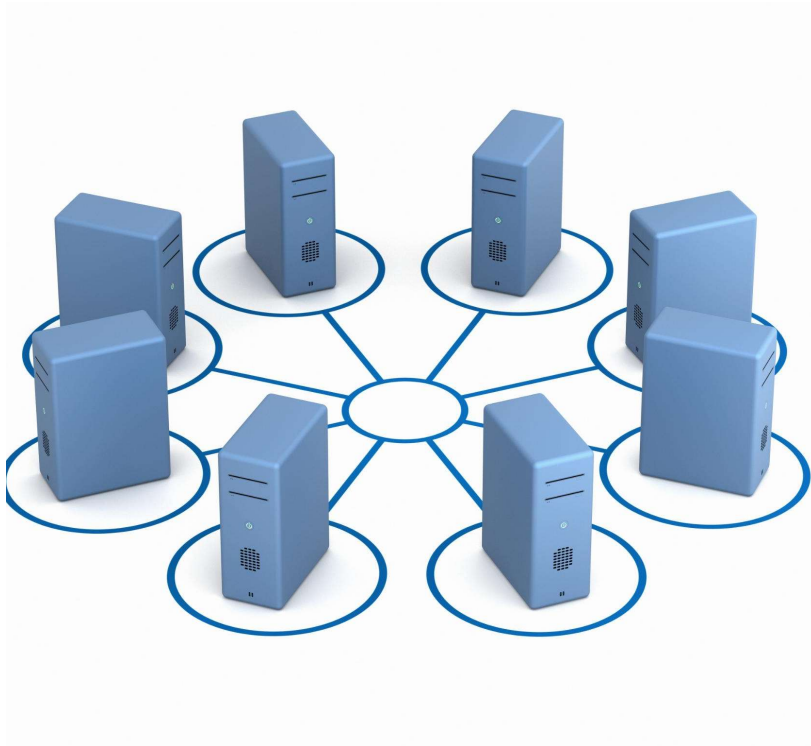
Event producers generate and dispatch events when specific actions or changes occur within a system or application.

Function of Event Consumers

Event consumers receive and process these events, often triggering additional actions or updates in response.



Event Brokers



Intermediary Communication Role

Event brokers serve as intermediaries, enabling communication between various systems or applications and improving information exchange.

Decoupling Producers and Consumers

They decouple event producers from consumers, allowing independent scaling and greater architectural flexibility.

Support Real-Time Data Streaming

Popular event brokers enable reliable, real-time data streaming and message delivery in distributed environments.



Event Broker - Apache Kafka

Distributed Streaming Platform

Kafka supports distributed event streaming, allowing data to flow seamlessly across clusters for reliable processing.

Real-Time Data Processing

Kafka enables real-time publishing, subscribing, and processing of event streams for instant insights and actions.

Scalable and Fault-Tolerant

Kafka helps organizations efficiently manage large volumes of data across applications by ensuring scalability and fault tolerance.

Cloud Event Specification

Standardized Event Description

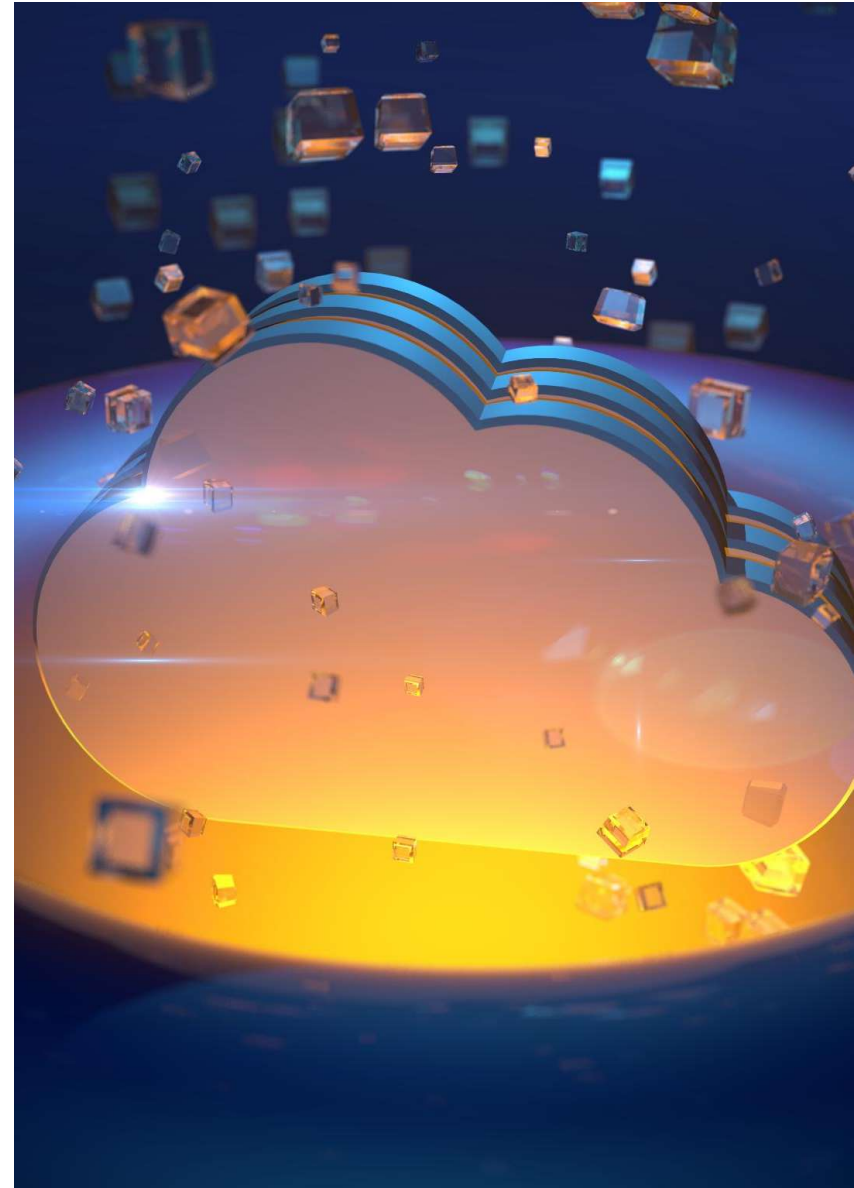
The Cloud Event Spec provides a unified format to describe event data, supporting interoperability across services.

Core Event Attributes

It defines common attributes like event type, source, and subject, ensuring consistency in event communication.

Flexible Transport Protocols

Supports multiple protocols for transporting events, enabling flexible and reliable cloud-native application workflows.



Cloud Event Specification Header v1.0

Name	Required	Description	Example
id		Identifies the event. Producers MUST ensure that source + id is unique for each distinct event.	63d6a150-c6a1-4c5b-bcc3-27d90c07941c
source		Identifies the context in which an event happened.	/default/sap.s4.beh/244572008
specversion		The version of the CloudEvents specification which the event uses.	1.0
type		Describes the type of the event	sap.s4.beh.businesspartner.v1.BusinessPartner.Changed.v1
datacontenttype		Content type of the value in data.	application/json

Standardized Event Metadata

CloudEvents Header v1.0 provides a specification for standardizing event metadata within HTTP requests, improving clarity and uniformity.

Required Header Fields

Headers like 'ce-specversion', 'ce-type', 'ce-source', and 'ce-id' are mandatory and ensure event interoperability across platforms.

Consistent Event Handling

These headers define event context, source, and identity, enabling reliable and consistent event integration in cloud-native systems.

Custom Headers

CloudEvents allows for custom metadata via extension attributes. These are additional fields that can be added to the event without violating the spec.



Events Documentation - Async API Specification (Swagger)

Event Schema Definition

Swagger enables defining event schemas and payloads, providing structure and clarity for event-driven systems.

For event-driven architectures **AsyncAPI** Specification is used

Clear Interface Contracts

Defining Interface contracts for event producers and consumers promotes interoperability and reduces misunderstandings.

Enhanced Team Collaboration

Swagger tools help teams generate, test, and validate event documentation, improving communication and reducing integration errors.

Batch Processing Pattern Overview

Note:

Performance aspect will be covered in the Performance Architecture Session

What is Batch Processing?

Definition of Batch Processing

Batch processing executes large volumes of data or transactions as grouped tasks at scheduled times.

Efficiency and Cost Savings

It enables efficient resource use and reduces costs by processing tasks when immediate action is unnecessary.

Limitations Compared to Real-Time

Batch processing provides delayed customer experience and is also not suitable for time-sensitive tasks like fraud detection or instant notifications.



Role in Core Banking



Core Processing

Batch processing efficiently handles large volumes of periodic banking tasks without disrupting real-time operations.

Key Process include:

Periodic Processes (e.g.. Interest accruals and capitalization, Statement Generation, Scheduled event processing),

File based processes (e.g.. Payment Files like from Clearing House, Salary Payments),

Mass Changes (e.g.. Index Rate Changes).



Scheduled Report Generation

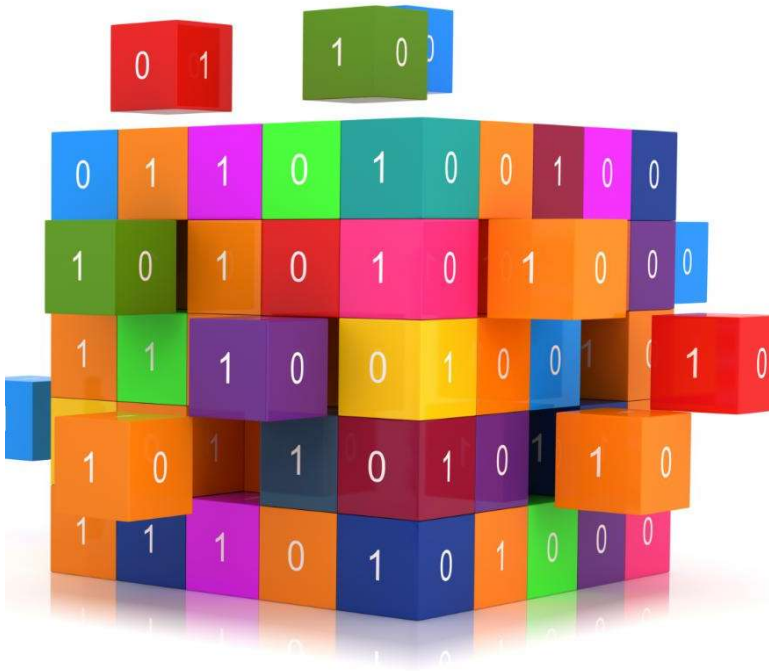
Batch jobs generate Day End Business Operational, Account and Transaction Analytics and Summaries at various levels (Account, Branch, Product, Business Unit, Geography etc..) during Day End / off-peak hours to maintain performance.



Regulatory Compliance Support

Batch processing ensures timely regulatory extracts and audit trails for compliance and reporting needs.

Batch Architecture Overview



Data Input Sources

Scheduled Event Logs, File Inputs, Activity Logs serve as inputs for batch processing in core banking.

Batch Processing Engines

Batch Processes are built on frameworks like **Spring Batch**, **Quarkus Batch** to transform, validate, process data efficiently.

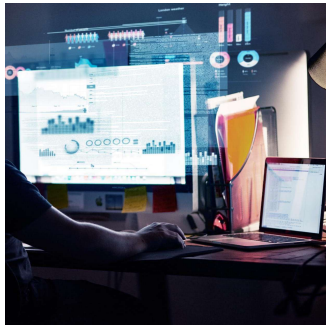
Storage and Output Modules

Relational databases and distributed file systems store processed data while output modules generate reports and update ledgers.

Scheduling and Error Handling

Jobs are Scheduled using Batch Schedulers to orchestrate job execution; error handling ensures validation, logging, and recovery strategies maintain reliability.

Batch Architecture - Checkpoint and Restart Basics



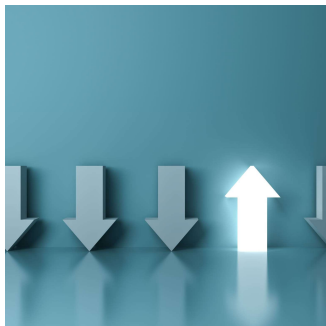
Saving Progress with Checkpoints

Batch processing systems periodically save the current state of jobs, creating checkpoints for future recovery if necessary.

All Batch Processes being built in Banking Technology Course Lab should have Checkpoint and Restart ability

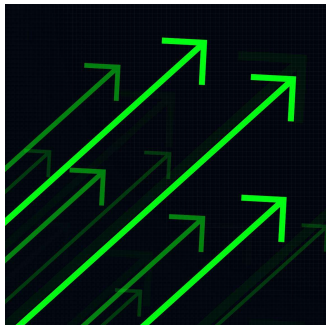
Make sure you include this while designing, building and testing. So need to do Checkpoint and have a Restart Key from where you can initiate your Batch Process after a failure

This will be part of your Lab assessment



Efficient Failure Recovery

Checkpoints allow systems to recover efficiently after a failure, minimizing data loss and avoiding a complete restart.



Resuming with Restart

The restart process uses the last saved checkpoint to continue operations, reducing downtime in large-scale data processing.

Batch Architecture - Intra Day Batch Processing

Multiple Daily Processing Cycles

Intra day batch processing runs several data cycles within a single business day, allowing more frequent updates and checks.

This is widely used in Banking Platforms

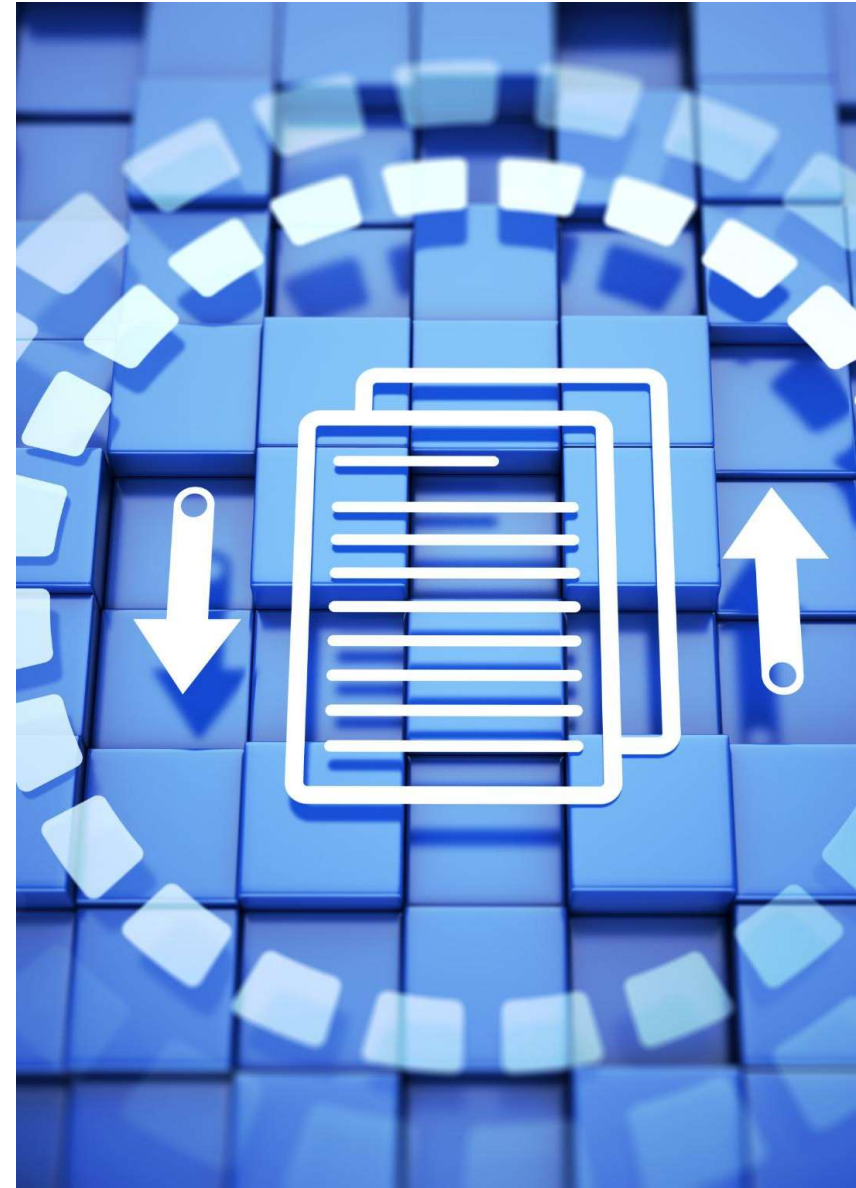
What is the Value

- Reduced Day End Processing Execution Time. This is critical in Banking as many Next Day Processes are dependent on previous day end completion
- Better End User Experience: Consumers get accurate business Responses and high availability
- Frequent record updates improve data accuracy as any exceptions can be caught earlier than waiting for day end, which is crucial for businesses.

What does it mean for Core Platform Design

Design the Banking Processes to be able to run multiple times in a day through the usage of Techniques like Periods and Timestamping the duration with a Period, so that Processes can be run multiple times in a day --

- Are you doing this in some of your Lab processes where this may make a meaning ?
- Discuss couple of processes where this could be useful



Batch Architecture - Batch Job Logs



Comprehensive Execution Records

Batch job logs store details such as job status, timestamps, and step outcomes for every execution.

Monitoring and Diagnostics

Logs are crucial for monitoring job progress and diagnosing failures, ensuring smooth data processing.

By reviewing the log, developers can easily trace errors and monitor job completion for better process management.

Concise Job Summaries

A summary provides an overview of number of records processed, successes, and failures for quick analysis.

Banking Tech Lab: In the Batch Processes design, development, testing include Batch Job logging

Batch Architecture - Schedulers

Automates Task Scheduling

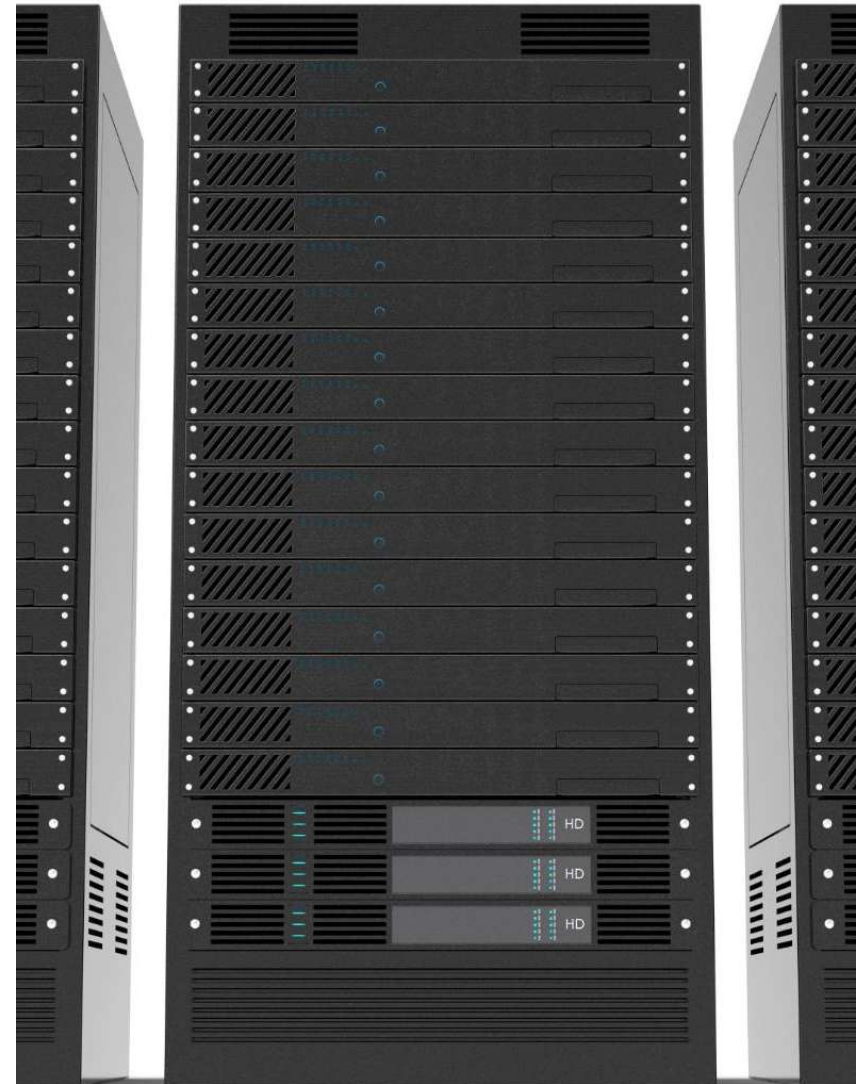
Batch schedulers automate repetitive job executions, improving efficiency and reducing manual interventions in computing environments.

Efficient Resource Management

Schedulers optimize resource usage by assigning jobs based on current availability, ensuring system performance and job completion.

Workflow and Job Prioritization

Tools like Airflow, Ctrl-M, Zena are vital for managing workflows and prioritizing jobs within complex computing systems.



Batch Architecture - Exception Handling

Error Detection and Logging

Effective batch processing detects errors and logs them to maintain reliability without stopping the process.

Skipping Faulty Records

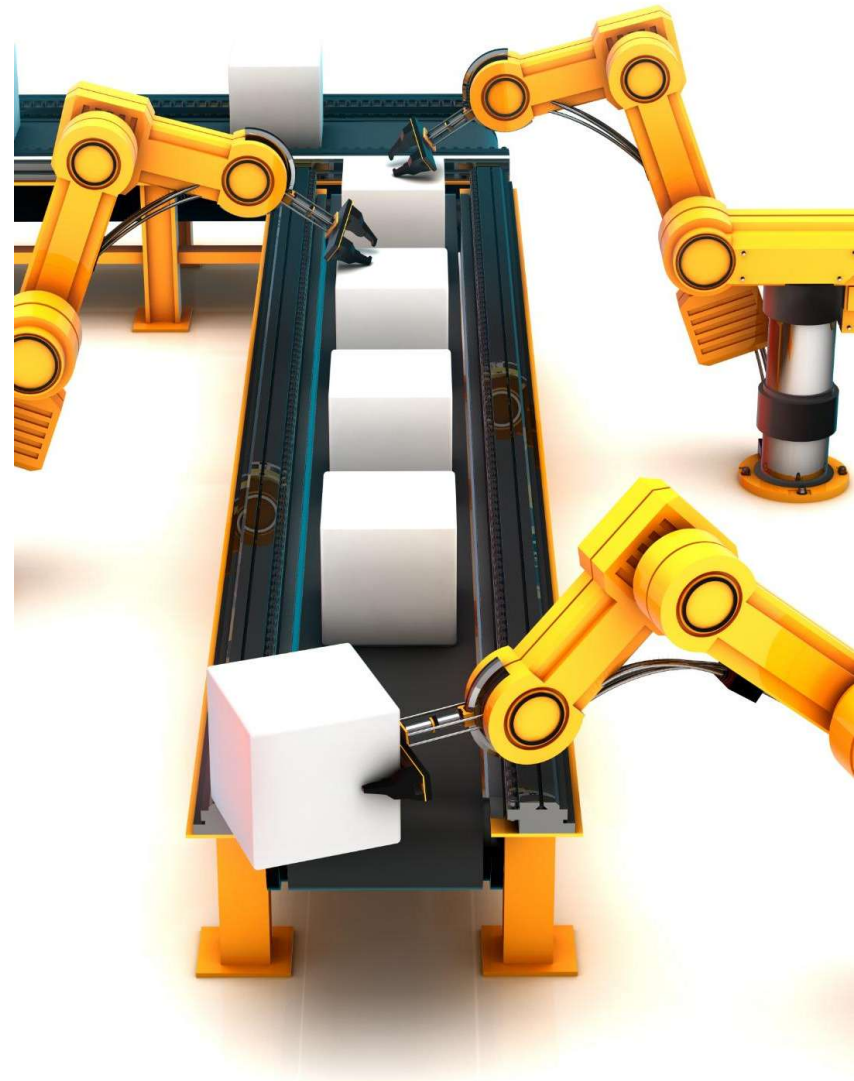
Faulty records can be skipped to prevent a single error from interrupting the entire batch process.

Retry and Error Reporting

Batch systems often retry failed steps and generate detailed error reports for troubleshooting and analysis.

Business Exception Queues

Business Exceptions are put in Exception Queue to be analyzed by a Human (Back office personnel) and determine the action to be taken



Advantages of Batch Processing

High Efficiency for Large Tasks

Batch processing efficiently handles high volumes of banking transactions without overwhelming system resources.

Cost-Effective Operation

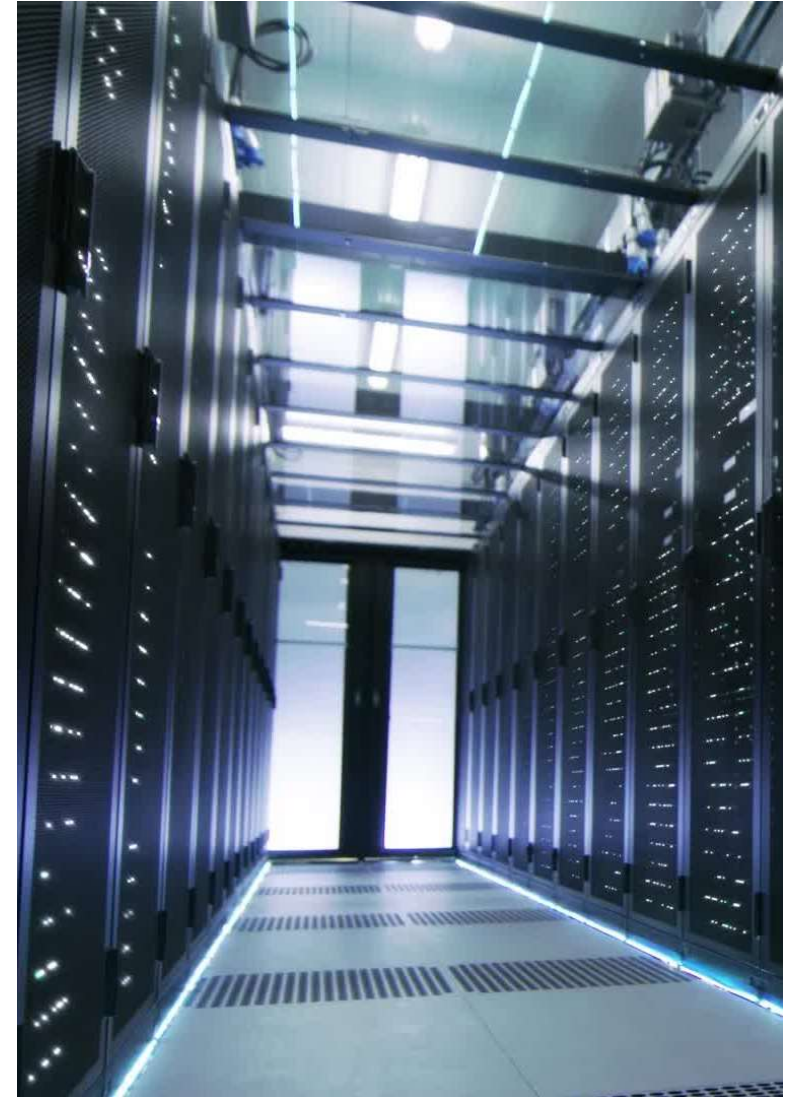
Running batch jobs during off-peak hours reduces infrastructure costs and optimizes resource use.

Simplified Error Handling

Grouped transactions facilitate easier error tracking, auditing, and reconciliation processes.

Supports Regulatory Compliance

Batch processing enables generation of audit trails and reports essential for banking compliance.





Limitations of Batch Processing

Lack of Immediacy

Batch processing cannot handle time-sensitive tasks requiring instant responses like fraud detection or notifications.

Latency in Scheduled Runs

Scheduled batch executions cause delays impacting data availability and user experience negatively.

Complex Error Recovery

Recovering from errors in batch systems is complex and requires robust, dependable mechanisms.

Challenges with Real-Time Data

Batch frameworks struggle to meet real-time data demands, limiting use in dynamic environments.

Trends and Evolution

Incremental Modernization

Banks prefer real-time or near real time capabilities for improved responsiveness, so trend is to move away from Batch Processing as much as possible and limit usage of Batch to where it is a must.

Hybrid Processing Models

Combining batch, real-time, and event-driven processing creates flexible and high-performance banking operations.

Cloud-Native Architectures

Containerization and Elastic design enable scalable, resilient batch operations in cloud environments.

AI-Driven Scheduling

Artificial intelligence optimizes job execution to enhance efficiency and responsiveness in batch processing.

